

Problem A1. Сан қосу техникасы: максимум

Формалды түрде, 1 цифрды қосу қажет, және алынатын санды барынша үлкейту керек. Әрине, 9-ды алдына қою тиімді.

Problem A2. Сан қосу техникасы: минимум

Формалды түрде, 1 цифрды қосу қажет, бірақ алынған санды барынша кішірейту керек. Қарапайым нұсқадан айырмашылығы, мұнда 0-ді санның басына қоюға болмайды — бұл шартқа қайшы, себебі сан өзгеріссіз қалады.

Маңызды, бірақ айқын факт! Цифр қосқаннан кейін санның ұзындығы әрдайым 1-ге артады. Келіңіз, 1 цифрын санның басына қоюдың тиімділігіне тоқталайық. Мысалы, егер бірінші цифр 2 болса, келесі өзгеріс болады:

$$2 \dots \rightarrow 12 \dots$$

Бұл жағдайда, бұл — ең тиімді нұсқа. Егер 1-ді басына қоймасақ, жаңа үлкен разрядта 2 қалады, ал бұл біз көрсеткен нұсқадан үлкен. Бұл логика бірінші цифр 1-ден үлкен кез келген сан үшін жұмыс істейді.

Бірақ егер бірінші цифр 1 болса, онда көшу былайша болады:

$$1 \dots \rightarrow 11 \dots$$

Бұл тиімді емес, себебі бұл 1 цифрын бірінші және екінші цифрдың арасына қойғанмен бірдей нәтиже береді. Мұндай жағдайда, 1-ді арасына қоюдың орнына, 0-ді қоюға болады, бұл анағұрлым тиімдірек болады. Міне, шешім осымен аяқталады.

Problem B. Сүйікті оқушы

Қатысушы i -дің ұпайлар жиынтығы a_i деп алайық. Барлық қатысушыларды ұпайларының кему реті бойынша реттейміз. Біз k -ші орында тұрмыз делік, онда бізден жоғары орында тұрғандардың ұпайын азайтуға тиіспіз. Қатысушы $i \in [1, k]$ әрбір тапсырма бойынша $b_1 \geq \dots \geq b_6$ (яғни берілген тапсырмаға жіберілген шешімдер арасындағы ең жоғарғы ұпай) деген ұпайларға ие болсын; оларды кему реті бойынша орналастырамыз. Қатысушының барынша аз тапсырмасын жою үшін ең үлкен ұпайы бар тапсырмадан бастаған тиімді. Сөйтіп, мынадай жағдайға жеткенше жоюды тоқтатпаймыз:

$$b_1 + \dots + b_j > a_k.$$

Мұндай алгоритм $O(n \log(n) + q)$ уақыт күрделілігімен жұмыс істейді, яғни орындалатын негізгі амалдар саны шамамен осындай (әдетте бір тұрақты коэффициентке тәуелді).

Problem C. Таңғалмастыру

Кез келген 23-таңғалмастырудың ұзындығы 400-ден көп. Сондықтан, $n \geq 23$ болғанда массив a таңғалмастыру бола алмайды, және жауап әрқашан «NO».

$n \leq 23$ үшін шешім. $dp[mask]$ деп, ұзындығы i префикс $mask$ -та жатқан элементтер үшін таңғалмастыру болып табылатын минималды i индексіні айтамыз. Егер біз маскаға j элементін қосатын болсақ, онда $dp[mask|2^j]$ күйін жаңартамыз, яғни $dp[mask|2^j] = \min(dp[mask|2^j], next[dp[mask]][j])$, мұнда $next[i][j]$ — a массивіндегі j саны орналасқан i -ден кейінгі бірінші позиция.

Егер толық маскадағы $dp \leq n$ болса, онда бұл массив таңғалмастыру болып табылады, әйтпесе жоқ. Жоқ алмастыруды табу үшін, әрбір $mask$ үшін біз соңғы рет $mask$ -қа қосқан j элементін сақтай аламыз.

Problem D. Матрицаны түрлендіру

Екі бүтін сан матрицасы A және B өлшемі $n \times m$ берілген, мұнда $n, m \geq 1$. A -ны B -ға айналдыру

үшін $n \cdot m$ операциясынан аспайтын қадамдармен орындау қажет:

$$\begin{aligned} A[x][y] &\leftarrow A[x][y] + c, \\ A[x][j] &\leftarrow A[x][j] - c, \\ A[i][y] &\leftarrow A[i][y] - c, \\ A[i][j] &\leftarrow A[i][j] + c, \end{aligned}$$

мұнда $x \neq i$, $y \neq j$ және $1 \leq c \leq 2 \cdot 10^9$.

Әр операцияның:

- кез келген жолдағы элементтердің қосындысын өзгертпейтінін;
- кез келген бағандағы элементтердің қосындысын өзгертпейтінін көрсетейік.

Шынында да, x жолында y бағанында c қосылып, j бағанында c алынып тасталады, жалпы өзгеріс $= 0$. i жолы, y бағаны және j бағаны үшін де осылай.

Осыдан $A \rightarrow B$ орындалуының қажетті шарты шығады:

$$\sum_{j=1}^m A[i][j] = \sum_{j=1}^m B[i][j] \quad (\text{барлық } i \text{ үшін}) \quad \text{және} \quad \sum_{i=1}^n A[i][j] = \sum_{i=1}^n B[i][j] \quad (\text{барлық } j \text{ үшін}).$$

Егер жолдар немесе бағандар қосындылары сәйкес келмесе, **жауап** бірден

–1.

Енді барлық жолдар мен бағандардағы A және B қосындылары сәйкес келетінін тексерейік. A -ны B -ға $n \times m$ операциясынан аспайтын түрде айналдыру үшін бұл жеткілікті екенін көрсетейік.

Идея. Бірінші $(n-1)$ жолдар мен $(m-1)$ бағандардағы барлық (i, j) элементтерін қарастырайық. Әр ұяшық үшін:

$$\text{diff} = B[i][j] - A[i][j].$$

Егер $\text{diff} \neq 0$, $A[i][j]$ -ны қажетті мәнге (қосып немесе алып тастап $|\text{diff}|$) түзететін **бір** операцияны қолданамыз, ал жанама өзгеріс тек соңғы жолдағы ($i = n$) және соңғы бағандағы ($j = m$) ұяшықтарға әсер етеді. Мысалы:

- егер $\text{diff} > 0$, онда

$$(x, y, i', j', c) = (i, j, n, m, \text{diff})$$

операциясы $A[i][j]$ -ны diff -қа арттырады;

- егер $\text{diff} < 0$, онда

$$(x, y, i', j', c) = (i, m, n, j, |\text{diff}|)$$

операциясын $A[i][j]$ -ны азайту үшін қолданамыз.

Осылайша, $1 \leq i \leq n-1$, $1 \leq j \leq m-1$ кезінде барлық (i, j) ұяшықтарындағы мәндерді «түзетеміз». Одан кейін $A[i][j]$ және $B[i][j]$ барлық жерде сәйкес келеді, тек соңғы жолда $i = n$ және соңғы бағанда $j = m$ тең болмауы мүмкін. Бірақ жолдар мен бағандар бойынша қосындылардың сәйкес келуіне байланысты (ал операциялар оларды өзгертпейді!) қалған ұяшықтар автоматты түрде B -дағы тиісті элементтермен тең болады.

Операциялар саны. Әр (i, j) ұяшығын «түзетуді» бір реттен артық жасамайтынымыз анық. Жалпы, максимум $(n - 1) \times (m - 1)$ операция, бұл $n \times m$ -нан аспайды.

Problem E. Кесіндіге қосу

Есепті шешу үшін $S_1, S_2, \dots, S_n$ жиындарын өздерімен бірге сақтамай, олардың әрбір x мәніне қатысты ақпаратты сақтаймыз. Атап айтқанда:

- **Бар болу аралықтары** — бұл x мәні бар болатын интервалдар.
- **Жоқ болу аралықтары** — бұл x мәні жоқ болатын интервалдар.

x мәнін $[l, r]$ аралығына қосу.

x санын $[l, r]$ аралығына қосқанда, бар болу және жоқ болу аралықтарын жаңарту қажет. Алдымен бар болу аралықтарын жаңартуды қарастырайық.

1. $[l, r]$ -мен қиылысатын барлық бар болу аралықтарын табу.
2. Бұл қиылысатын аралықтарды $[l, r]$ -мен біріктіріп, бірыңғай диапазонға келтіру.
3. Тізімнен ескі қиылысатын аралықтарды өшіріп, жаңадан біріктірілген аралықты қосу.

Мысалы, егер x үшін ағымдағы бар болу аралықтары $[2, 3]$, $[6, 8]$ және $[10, 12]$ болса, ал қосылатын аралық $[5, 10]$ болса, жаңартудан кейінгі аралықтар $[2, 3]$ және $[5, 12]$ болады.

Жоқ болу аралықтарын жаңарту да осыған ұқсас түрде орындалады:

- $[l, r]$ -мен қиылысатын жоқ болу аралықтарын өшіреміз.
- Егер жоқ болу аралықтары $[l, r]$ -мен ішінара қиылысса, оларды $[l, r]$ -мен қиылыспайтын бөліктерге бөліп тастаймыз.
- Тізімді жаңадан қалған бөліктерімен жаңартамыз.

Сұрауларға жауаптар.

- **Қиылысу:** егер $[l, r]$ аралығын толығымен қамтитын бар болу аралығы болса, онда x мәні $[l, r]$ ішіндегі барлық жиындардың қиылысуына кіреді.
- **Біріктіру:** егер $[l, r]$ аралығын толығымен жабатын жоқ болу аралығы болмаса, онда x мәні $[l, r]$ ішіндегі барлық жиындардың бірігуіне кіреді.

Аралықтарды жылдам есептеу.

$[l, r]$ аралығының $[L, R]$ диапазонын толығымен қамти алатынын тексеру үшін мынадай идеяны қолдануға болады.

Әрбір $[l, r]$ аралығын (x, y) нүктесі ретінде қарастырамыз, мұндағы:

- $x = l$ — аралықтың сол жақ шекарасы.
- $y = r$ — аралықтың оң жақ шекарасы.

Мәселе (x, y) нүктелерінің санын табуға тіреледі, олардың барлығы мынадай тікбұрышты аймақта жатуы қажет: $0 \leq x \leq L$ және $R \leq y \leq n$

Осы мақсатта Fenwick Tree немесе Segment Tree секілді екі өлшемді деректер құрылымдарын пайдалануға болады, олар тікбұрыштардағы нүктелерді санау мен жаңа нүктелерді қосуды тиімді орындайды.

Қорытынды шешім.

- x мәнін $[l, r]$ аралығына қосу үшін:
 - Бар болу аралықтарын жаңартамыз: $[l, r]$ -мен қиылысатын аралықтарды біріктіреміз.
 - Жоқ болу аралықтарын да ұқсас түрде жаңартамыз.
- Қиылысу үшін $[l, r]$ аралығын қамтитын бар болу аралықтарының санын есептейміз.
- Біріктіру үшін $[l, r]$ аралығын қамтитын жоқ болу аралықтарының санын n -нен шегеріп тексереміз.

Problem F. Ертегі қаласы

Графтың әрбір байланысқан компонентасын жеке-жеке қарастырайық.

Жағдай: компоненттегі қырлардың саны жұп. Егер қырлардың саны жұп болса, онда графты ұзындығы 2 болатын жолдармен жабуға болады. Ол үшін:

1. Кез келген төбеден DFS жібереміз, атасын сақтай отырып.
2. Әр қадамда:
 - Барлық кері қырларды және «артық» төменгі қырларды жинаймыз.
 - Ұзындығы 2 болатын жолдармен жабу үшін қырлар жұбын құрамыз.
3. Егер «артық» қыр қалса, оны атасына төбеден алынған жоғары қырмен жұптастырамыз.
4. Егер «артық» қыр болмаса, қырды жоғары қарай береміз, ата төбе оны өңдеу үшін.

Қырлардың саны жұп болғандықтан, әрқашан дұрыс бөліністі табуға болады.

Алгоритмді түсіну үшін DFS-tree тақырыбын зерттеуді ұсынамыз.

Жағдай: компоненттегі қырлардың саны тақ. Егер қырлардың саны тақ болса, екі жағдай мүмкін: компонента данақ болып табылады немесе ол циклды қамтиды:

Жағдай: компонента циклды қамтиды.

1. Компонентадағы циклды табамыз. Цикл төбелерінің реттілігі $c_1, c_2, \dots, c_k$ болсын.
2. Бір қырды (c_1, c_k) жоямыз. Бұдан кейін қалған компонента байланысқан болып, қырлардың саны жұп болады. Ұзындығы 2 болатын жолдармен жабу үшін жоғарыда сипатталған алгоритмді қолданамыз.
3. Енді жойылған қырды бір жолға қосамыз:
 - Егер c_1 төбесін аяқтайтын жол бар болса, жойылған қырды сол жолға қосамыз.
 - Егер ондай жол болмаса, (c_2, c_1) қырын қамтитын жолды қарастырамыз. Ол жол $[c_2, c_1, A]$ болсын. Бұл жолды $[c_k, c_1, A]$ етіп қайта құрамыз, ал жойылған қыр (c_2, c_1) болады.
 - Жаңа жойылған қыр үшін процесті қайталаймыз. Жаңа жойылған қыр (c_3, c_2) болса, онда c_3 төбесін қамтитын жолдарды қарастырамыз.
4. Егер жойылған қыр (c_k, c_{k-1}) болса, оны $[c_k, c_1, A]$ жолына қосамыз.

Жағдай: компонента данақ болып табылады.

- Егер барлық төбелердің дәрежесі тақ болса, шешім жоқ. Данақты кез келген төбеге іліп қоямыз. Түбір болмайтын төбенің барлық балалары жапырақ болады. Олардың саны жұп болады, оларды жұптарға бөліп, жоямыз. Сонда бұл төбе де жапырақ болады. Бұл процесс түбірде тақ санды жапырақ-балалар қалғанға дейін жалғасады, оларды жабу мүмкін болмайды.
- Егер жұп дәрежелі төбе болса, шешім бар. Данақты сол төбеге іліп қоямыз. Алгоритм:
 1. Әрбір түбірдің баласының қосалқы данағын ұзындығы 2 болатын жолдарға бөлу алгоритмін іске қосамыз.
 2. Әрбір баланың қосалқы данағында бір қыр ғана қалуы мүмкін. Сонда түбірден ұзындығы 1 және 2 болатын жолдар шығады.
 3. Ұзындығы 1 болатын жолдардың саны тақ болады. Ұзындығы 2 болатын жолдардың саны да тақ болады.
 4. Ұзындығы 1 болатын бір жолды және ұзындығы 2 болатын бір жолды біріктіріп, ұзындығы 3 болатын жол жасаймыз. Осыдан кейін ұзындығы 1 болатын жолдардың саны жұп болып, оларды жұптарға бөлуге болады, ал ұзындығы 2 болатын жолдарды сол күйінде қалдырамыз.